Framework Selection Report

Student: GUNTANG, MORRIS JINN R.

1. Introduction

This report compares React + Next.js 14 (App Router) and Vue + Nuxt 3/4-era tooling for the Pangasinan Heritage Digital Showcase. The target system is a small, mobile-first, static tourism website that must remain accessible, maintainable, and suitable for GitHub Pages deployment.

2. Project Requirements

- Fast loading on mobile networks
- Atomic Design component structure
- Static generation and GitHub Pages readiness
- Beginner-friendly code that is easy to explain
- Accessible semantic HTML and responsive layouts

3. Frameworks Compared

3.1 Next.js 14 / React

Next.js 14 uses the App Router inside the `app` directory and supports layouts, nested routes, Server Components, and static export via `output: 'export'`.

3.2 Nuxt 3 / Vue

Nuxt provides file-based routing, component auto-imports, SSR by default, and static generation with `nuxt generate`. The official GitHub Pages deployment guide also documents the `github_pages` preset.

4. Evaluation Method

The score for each criterion uses a 1-5 scale, where 5 means the framework is the strongest fit for this specific assignment. Weighted totals are computed as `weight × (score / 5)`. Quantitative claims are limited to official documentation, GitHub repository data, and local benchmark measurements taken on 2026-09-01 using fresh official starters under Node.js 24.18.0.

> **Important measurement note:** the two frameworks do not report identical build metrics. Next.js reports route-level first-load JavaScript, while Nuxt exposes chunk and generated-output sizes. The comparison therefore uses those official outputs carefully and supplements them with local install footprint and requirement fit.

5. Quantitative and Weighted Comparison

Criterion	Weight	Nuxt Score	Next.js 14 Score	Evidence / Method
Bundle / performance	25	4	4	Local starter benchmark on 2026-09-01: Nuxt starter `node_modules` was 168,715,739 bytes and generated a 231,597-byte static output. Next.js 14 starter `node_modules` was 230,855,749 bytes, while `next build` reported 92.7 kB first-load JS for `/`. Result: both are performant enough for a small static showcase, with Nuxt showing the lighter local setup footprint and Next showing a smaller reported default first-load JS.
Developer velocity	20	5	4	Nuxt documentation emphasizes writing `.vue` files immediately with auto-imports, zero-config TypeScript, file-based routing, and configured build tooling. That matched this project well: the showcase was assembled directly from SFCs without manual component imports in every page.
Ecosystem maturity	10	4	5	GitHub repository metrics on 2026-09-01 showed `nuxt/nuxt` at 60,807 stars and `vercel/next.js` at 142,054 stars. Both are
Total	**100**	**91.0**	**82.0**	**Weighted total = sum of weight × (score / 5)**

Criterion	Weight	Nuxt Score	Next.js 14 Score	Evidence / Method
				mature; Next.js has the larger ecosystem signal.
Learning curve	10	5	3	Nuxt presents a beginner-friendly single-file component model. By contrast, Next.js 14 App Router documentation introduces additional React-specific concepts such as Server Components, Client Components, layouts, loading states, and multiple special files.
Component architecture	15	5	4	The assignment requires Atomic Design and reusable UI building blocks. Vue single-file components keep template, script, and style together, which made atoms, molecules, and organisms straightforward to organize and document.
Documentation / community support	10	4	5	Both frameworks have strong official documentation. Next.js benefits from a broader React community footprint, while Nuxt provides focused official guidance for routing, rendering, and GitHub Pages deployment.
Project suitability	10	5	4	This project is a small, content-driven, static tourism showcase. Nuxt directly supports `nuxt generate`, file-based routing, component auto-imports, and a GitHub Pages deployment preset, which aligns closely with the assignment requirements and the final local build that produced a working 277,032-byte static site.
Total	**100**	**91.0**	**82.0**	**Weighted total = sum of weight × (score / 5)**

6. Qualitative Comparison

Next.js 14 is excellent for React teams and has strong ecosystem maturity. However, the Pangasinan showcase does not need complex server features. The assignment prioritizes a clean component hierarchy, beginner readability, and a simple static deployment path, all of which align closely with Nuxt's conventions.

7. Selected Framework

Selected framework: Vue + Nuxt.

8. Project Suitability

The final local implementation confirms the selection. The Nuxt project renders five destinations, filters them on the client, generates static output successfully, and includes a dedicated component showcase page for documentation. Browser verification also confirmed the required 1 / 2 / 3-column grid behavior across mobile, tablet, and desktop breakpoints.

9. Conclusion

Both frameworks are capable of delivering a high-quality static showcase. Next.js 14 remains a strong option, especially for established React teams, but Nuxt is the better fit for this assignment because it offers a gentler learning curve, clean single-file component organization, and a direct path to the required static deployment workflow.

10. References

1. Nuxt Introduction: https://nuxt.com/docs/4.x/getting-started/introduction
2. Nuxt GitHub Pages Deployment: https://nuxt.com/deploy/github-pages
3. Next.js 14 Routing Fundamentals: https://nextjs.org/docs/14/app/building-your-application/routing
4. Next.js 14 Static Exports: https://nextjs.org/docs/14/app/building-your-application/deploying/static-exports
5. GitHub repository data for `nuxt/nuxt` and `vercel/next.js`, captured on 2026-09-01.
6. Local benchmark measurements stored in the session workspace for fresh Nuxt and Next 14 starters.

Project static verification note: the final Pangasinan site generated successfully, and its current prerendered `index.html` file is 17,408 bytes within a 277,032-byte static output directory.